

QueryQuest: Assignment 1

Vibha Narayan

7th June 2026

1 Assignment Solutions

Part 1: Short Answer Questions

1. **What is the difference between a Database Management System (DBMS) and a traditional file system? Provide one concrete scenario in which a DBMS is clearly the better choice and explain your reasoning.**

A DBMS (Database Management System) is a software system that manages, stores, and retrieves data efficiently in a structured format. It acts like a bridge between a central database and multiple clients. It allows users to create, update, and query databases efficiently while reducing data redundancy and inconsistency through centralized control.

In contrast, traditional file systems manage data using isolated files on hard drives. This setup inherently leads to problems like data redundancy, difficult access, and data inconsistency due to conflicting, outdated information stored across different files.

Concrete Scenario: Consider a university management setup. In a traditional file-based system, separate folders or spreadsheets are maintained for *Academics*, *Results*, and *Hostel Details*. If an administrator needs to check whether a specific student has paid their mess bills, they would have to open the specific "Mess Bill" file and manually scroll through it page-by-page, consuming significant time and effort.

By using a DBMS, all this data is centralized and linked. The user can simply execute a quick search query against the student's unique ID or name to instantly retrieve their payment status, demonstrating how a DBMS drastically optimizes data retrieval and saves time.

2. **In an ER diagram, what distinguishes a weak entity from a strong entity? How is the primary key of a weak entity formed and what is the identifying relationship associated with it called?**

A *Strong Entity* has a key Attribute that can uniquely identify each instance of the entity. A Strong Entity does not depend on any other Entity in the Schema for its identification. It has a primary key that ensures its uniqueness and is represented by a rectangle in an ER diagram whereas a *Weak Entity* cannot be uniquely identified by its own attributes alone. It depends on a strong entity to be identified.

The relationship between the weak entity type and its identifying strong entity type is called identifying relationship and it is represented by a double diamond.

3. **A university database stores the phone numbers of students, where a student may have multiple phone numbers. Should the phone number be modeled as a simple attribute, a composite attribute or a multivalued attribute? Justify your answer.**

Phone number should be modelled as *multivalued attribute* (consists of more than one value for a given entity), as a student can have multiple phone numbers.

4. Explain the difference between a candidate key and a super key.

Consider the relation: Student(roll no, email, name, branch)

List all candidate keys and provide at least two super keys that are not candidate keys.

The minimal set of attributes that can uniquely identify a tuple is known as a *candidate key*.

A candidate key is a minimal super key with *no redundant attributes*. It must contain unique values, ensuring that no two rows have the same value in the candidate key's columns.

The set of one or more attributes (columns) that can uniquely identify a tuple (record) is known as *Super Key*. It may *include extra attributes* that aren't important for uniqueness but still uniquely identify the row.

Candidate Key: Roll No & Email

Super Key: Any combination of Candidate + Non-Candidate Columns

For eg: Roll No, Branch & Email, Name

5. Suppose a row is inserted into an Enrollment table with a student id value that does not exist in the Student table.

- Name the constraint that is violated.
- Explain what it protects against
- State what should happen to the insert operation.

Referential Integrity Constraint is violated here as the new student_id is only inserted in the ENROLLMENT table which is the foreign key for this table but S_ID is the primary key for the STUDENT table and it does not have the newly inserted student_id.

It protects against **orphaned records** and data inconsistency. In this scenario, it prevents the database from creating an enrollment record for a student who does not actually exist in the university's system, ensuring that all relational links between tables remain valid.

The database management system (DBMS) will automatically **reject/abort the insert operation** and throw an error. The state of the database will remain unchanged, ensuring no invalid data is saved.

6. When mapping a many-to-many (M:N) relationship from an ER diagram to a relational schema:

- What additional structure must be created?
- What does its primary key consist of?
- Why can the relationship not simply be represented as a column in one of the participating entity tables?

A distinct, separate relationship table must be created (for example, an Enrolls table linking the Student and Compulsory_Courses entities).

Because the primary key attributes from both participating tables repeat individually inside this relationship table, the primary key must consist of a unique **combination** of both keys. This creates a **composite primary key** consisting of the foreign keys of both participating entities (e.g., {S-Id, C-Id}).

The relationship can not be simply represented as a column in one of the participating entity tables both entity keys (S-Id and C-Id) repeat because a single student can enroll in multiple compulsory courses, and a single compulsory course can have multiple students.

Part II: Long Answer Question — Online Food Delivery Platform

Part A – Entity and Relationship Identification

1. Entities and Key Attributes

Based on the scenario, we can identify four core entities. Their unique primary key attributes are underlined:

- **Customer:** {customer_id, name, email, phone_number, address}
- **Restaurant:** {restaurant_id, name, cuisine_type, location, rating}
- **Order:** {order_id, order_date, total_amount, delivery_status}
- **Menu Item:** {item_id, name, description, price}

2. Relationships and Cardinality Justifications

Below are the structural relationships connecting these entities:

a) Customer to Order

- **Cardinality:** $1 : N$ (One-to-Many)
- **Justification:** A single customer can place multiple orders over time, but each specific order belongs to exactly one customer.

b) Restaurant to Order

- **Cardinality:** $1 : N$ (One-to-Many)
- **Justification:** A restaurant can receive and fulfill multiple orders, but any single order is placed with and fulfilled by exactly one restaurant.

c) Restaurant to Menu Item

- **Cardinality:** $1 : N$ (One-to-Many)
- **Justification:** A restaurant can offer multiple distinct menu items, but each specific menu item profile belongs uniquely to that restaurant's menu.

d) Order to Menu Item

- **Cardinality:** $M : N$ (Many-to-Many)
- **Justification:** An order can contain multiple different menu items, and a single menu item can be a part of many different orders.
- **Relationship Attribute:** *quantity* (Since quantity depends on both the specific order and the specific item, it belongs as an attribute on this relationship).

Part B – Attribute Classification

1. Multivalued Attribute: *phone_number* (or *cuisine_type*)

Explanation: An attribute qualifies as multivalued when it can hold more than one distinct value for a single entity instance simultaneously. For instance, a **Customer** may have multiple active phone numbers (e.g., a personal mobile number and a work number), or a **Restaurant** might specialize in multiple cuisines (e.g., *Italian* and *Fast Food*) at the same time.

2. Composite Attribute: *address* (or *name*)

Explanation: An attribute qualifies as composite when it can be broken down into smaller, independent sub-attributes that have their own individual meaning. For example, instead of storing a customer's **address** as a single long string, it is naturally composed of smaller components: *Street Address*, *City*, *State*, and *Zip Code*. Dividing it this way allows the database to easily sort or filter records by specific regions.

Part C – Relational Schema Design

To map the many-to-many ($M : N$) relationship between **Order** and **MenuItem**, we must introduce a new junction table. Following the standard relational schema notation, the table design is structured as follows:

Order_MenuItem_Bridge(order_id, item_id, quantity)

Schema Specifications:

- **Primary Key:** The primary key is a composite key consisting of the combination of (`order_id`, `item_id`). This ensures that a single menu item cannot be added as a duplicate row within the same order; instead, its amount is updated.
- **Foreign Keys:**
 - `order_id` references the `order_id` column in the **Order** table.
 - `item_id` references the `item_id` column in the **MenuItem** table.
- **Additional Attributes Required:**
 - `quantity`: This relationship attribute is structurally required here because the amount of a specific food item ordered depends directly on both the specific order and the specific item selected.

Part III: ER Diagram Design

Question 1: Hospital ER Diagram

Below is the designed Entity-Relationship (ER) diagram for the hospital management scenario. It represents the core entities, their respective attributes, primary keys, and structural participation constraints:

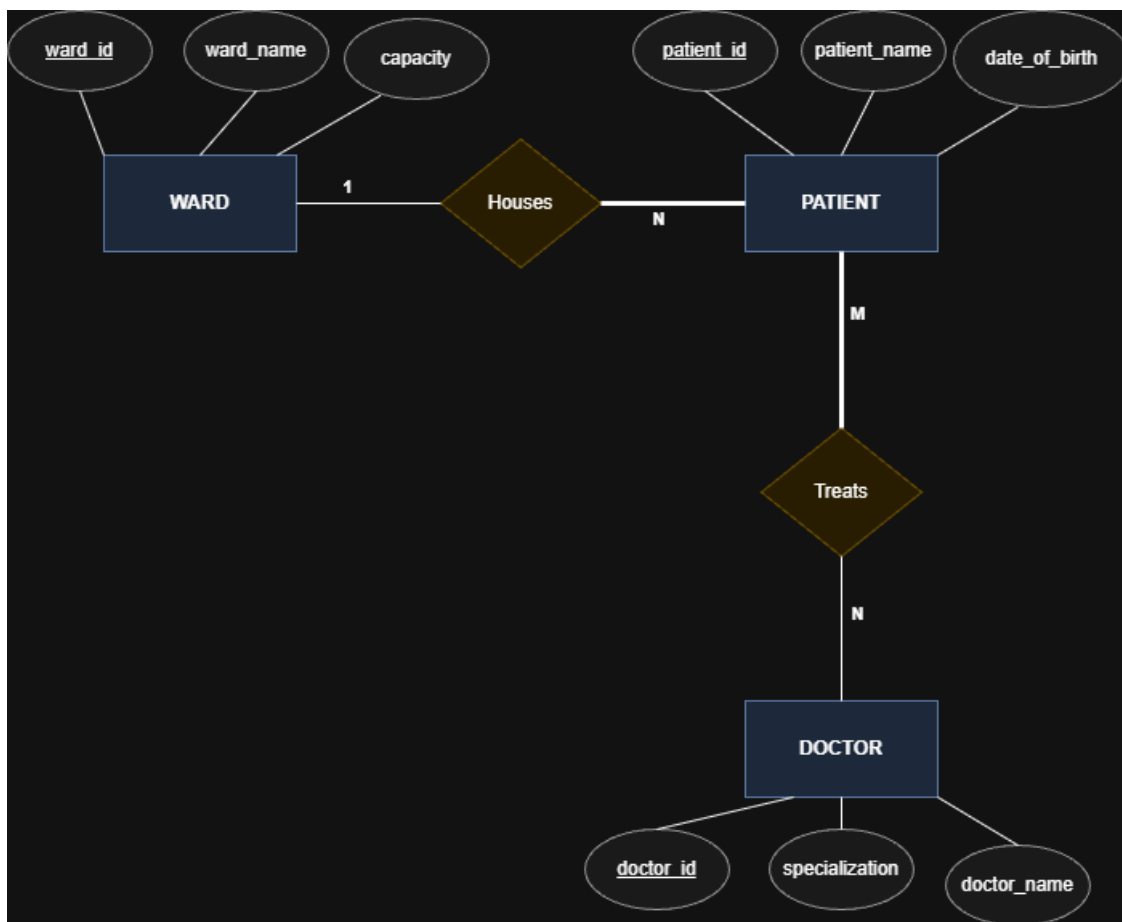


Figure 1: Hospital ER Diagram

Question 2: Design Justification

The **Doctor–Patient** relationship is modeled as $M : N$ (Many-to-Many) because a single patient can be treated by multiple specialist doctors during a hospital stay, and a single doctor simultaneously manages a roster of multiple patients. In a relational schema, this relationship collapses into an **associative entity** (junction table) typically named **Treatment**, whose composite primary key consists of the combination of foreign keys from both parent entities: (**patient_id**, **doctor_id**).

Regarding participation constraints, the **Ward–Patient** (*Houses*) relationship exhibits **partial participation** on the Ward side, as a ward can structurally exist within the hospital system even if it currently contains zero patients. Conversely, the Patient side has **total participation** because an admitted inpatient cannot exist in the tracking database without being assigned to a physical bed or ward. Similarly, in the **Patient–Doctor** (*Treats*) relationship, the Patient exhibits **total participation** because every admitted individual must have at least one primary physician responsible for their clinical care to maintain medical accountability.

For the attribute design, **date_of_birth** was modeled as a simple, atomic attribute rather than storing a calculated “Age” column. This decision avoids a stale data anomaly, as an explicit age column changes daily and would render the database inaccurate over time. In contrast, a static date of birth allows the system to dynamically calculate a patient’s precise age whenever a query is run.